รหัส:_____

ชื่อ-สกุล:_____

Interrupts

1. ให้เขียน Interrupt Vector ของ ET-8032 Board และ Single Chip

Interrupt Source	Single Chip 8052	ET-8032 V2 Board	Priority
Power On Reset	0000H	-	0
External Interrupt 0			1
Timer 0 Interrupt			2
External Interrupt 1			3
Timer 1 Interrupt			4
Serial Interrupt			5
Timer 2 Interrupt			6

IE: interrupt Enable Register

EA	-	ET2	ES	ET1	Ex1	ET0	EX0

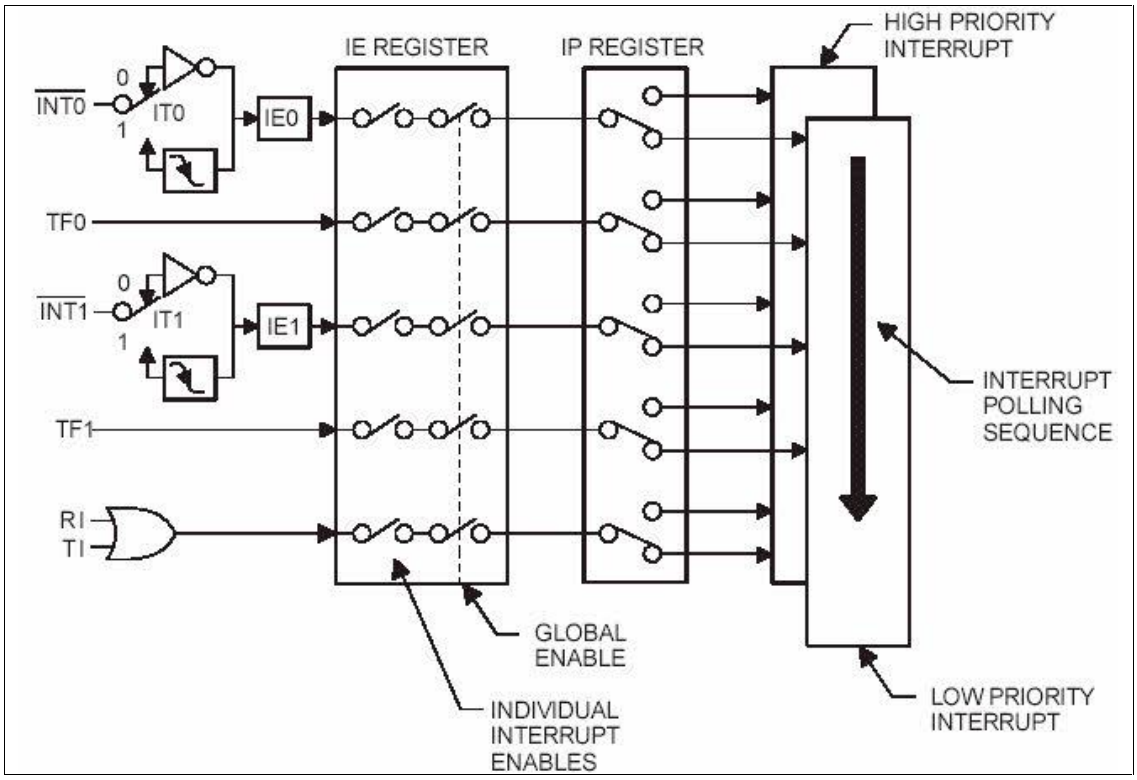
In Case of External Interrupts, Select Interrupt Type Control Bit

ITx=0: Low level trigger

ITx=1: Low level **edge** trigger

IP: interrupt Priority

-	-	PT2	PS	PT1	PX1	PT0	PX0



2. การใช้งาน Interrupts ด้วยโปรแกรมภาษาซี

```
#include <reg52.h>
sbit CS_Seg0 = P1^4;
sbit CS_Seg1 = P1^5;
#define PortO_DispData P0
unsigned int Count=0;
code unsigned char CodeSeg[] =    {       0x77,0x11,0x6D,0x5D,0x1B,0x5E,0x7E,0x15,
                                          0x7F,0x5F,0x3F,0x7A,0x66,0x79,0x6E,0x2E };
//==================================================
void Delay(unsigned char msec)
        {       unsigned char i,j;
                while(msec)
                {       for(i=1;i<=250;i++)                //not :
                                for(j=1;j<=113;j++);       //with ;
                        msec--;
                }
        }
//==================================================
void int0_isr(void) interrupt 0
        {       EX0 = 0;        // Disable Interrupt INT0
                Count++;        // Increment Counter
                Delay(4);       // Debounce Signal
                EX0 = 1;        // Enable Interrupt INT0
                IE0 = 0;
        }
//==================================================
main()
        {       IT0 = 0;        // Edge Trigger Interrupt
                EX0 = 1;        // Enable External 0 Interrupt
                EA = 1;         // Global Interrupt Enable
                while (1)
                        {       Count = Count&0x0f;
                                CS_Seg0 = 0;          CS_Seg1 = 0;
                                PortO_DispData = ~CodeSeg[Count];
                        }
        }
```

การใช้งาน Interrupts อื่นสำหรับโปรแกรม C

Interrupt	Name	Numbers used by 8051 C
External Interrupt 0	(INT0)	0
Timer Interrupt 0	(TF0)	1
External Interrupt 1	(INT1)	2
Timer Interrupt 1	(TF1)	3
Serial Communication	(RI + TI)	4
Timer 2 (8052 only)	(TF2)	5

Chapter 11

Interrupts Programming

Outlines

- Contrast and compare interrupts VS polling
- Explain the purpose of the ISR
- List the 6 interrupts of the 8051
- Explain the purpose of the interrupt vector table
- Enable or disable 8051 interrupts
- Program the 8051 timers using interrupts
- Describe the two external hardware interrupts of the 8051

Outlines(2)

- Contrast edge-triggered with level-triggered interrupts
- Program the 8051 for interrupt-based serial communication
- Define the interrupt priority of the 8051

Header

H1: 8051 Interrupts
- Interrupts Vector
- ISR: Interrupts Services Routine
- Register IE

H2: Timer Interrupts

H3: External Interrupts
- Bit ITx: Interrupt Type in TCON
  (0=Lo Lelev,1=Falling Adage)

H4: Serial Interrupts

H5: Interrupts Priority
- Register IP

H1: 8051 Interrupts

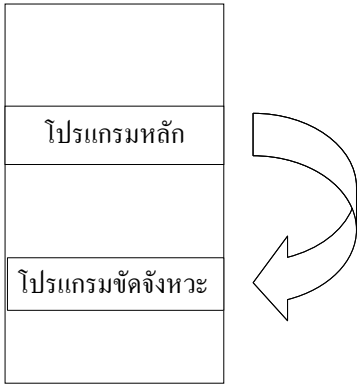
H1.1:Interrupts Program

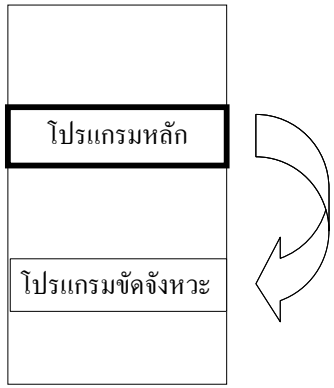
เทคนิควิธีในการเขียนโปรแกรม

1. Polling : เช่นกรณีรับสัญญาณจากพอร์ตอนุกรมวิธีการนี้จะใช้การวนตรวจสอบเงื่อนไขอยู่ตลอดเวลาเพื่อตรวจสอบว่ารับข้อมูลครบหรือยังด้วยการใช้ JNB RI,$

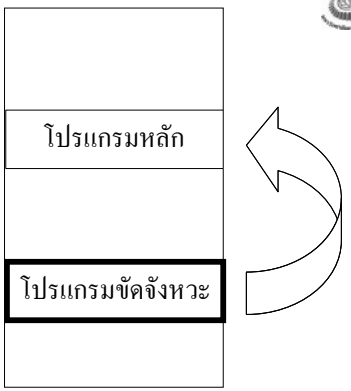
2. Interrupt : วิธีการนี้เราไม่ต้องเสียเวลามาตรวจสอบ แต่เราจะใช้การขัดจังหวะ (Interrupt) เมื่อถึงสถานะที่ต้องการ เช่น เกิดการ Overflow ของบิต RI

<u>การขัดจังหวะ(Interrupt)</u> คือการที่มีสัญญาณหรือคำสั่งที่ไม่ใช่คำสั่ง CALL หรือ JMP และทำให้การทำงานของโปรแกรมหลักถูกขัดจังหวะ และกระโดดไปยังตำแหน่ง ใดตำแหน่งหนึ่งที่กำหนดไว้และเมื่อทำงานเสร็จสิ้นก็จะกระโดดกลับมายังโปรแกรมหลักและ ทำงานต่อไป





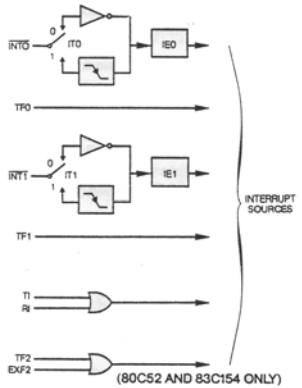
เมื่อเกิดการ Interrupt



หลังการ Interrupt

Interrupt

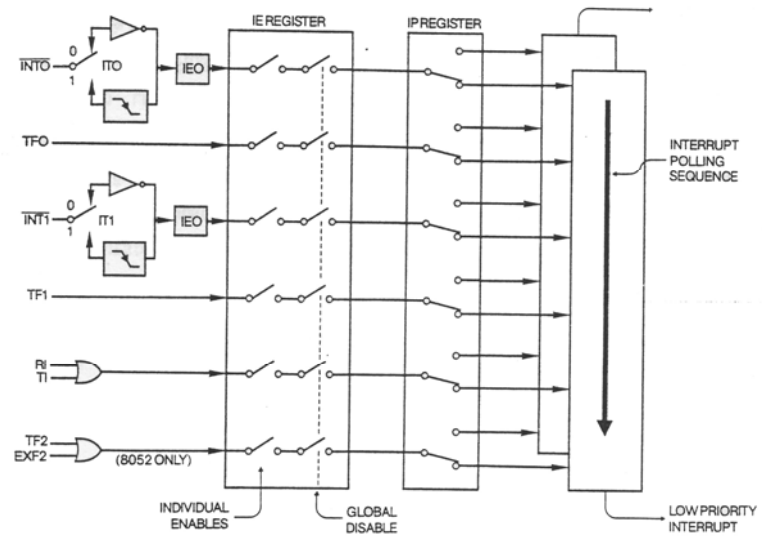
สำหรับตำแหน่งที่โปรแกรมจะกระโดดไปทำงานเมื่อเกิดการขัดจังหวะ (Interrupt) เราจะเรียกว่า *<u>Interrupt Vector</u>* และโปรแกรมที่จะทำงานเมื่อเกิดการ Interru เราจะเรียกว่า *<u>Interrupt Service Routine</u>* กรณีของ MCS-51 จะมีแหล่งกำเนิดสัญญาณ Interrupt ได้ทั้งหมด 5 แหล่ง (6 แหล่งสำหรับ 8052) ดังรูป



ตำแหน่งของ Interrupt Vector ของแหล่งกำเนิด Interrupt แต่ละแหล่งจะมีค่าตามตาราง

Interrupt Source	Single Chip 8052	ET-8032 V2 Board	Priority
Power On Reset	0000H	-	0
External Interrupt 0	0003H	9F3FH	1
Timer 0 Interrupt	000BH	9F42H	2
External Interrupt 1	0013H	9F45H	3
Timer 1 Interrupt	001BH	9F48H	4
Serial Interrupt	0023H	9F4BH	5
Timer 2 Interrupt	002BH	9F4EH	6

รูปแสดงการทำงานของระบบการขัดจังหวะใน MCS-51



H1.2: การตั้งค่า Register ที่เกี่ยวข้องเพื่อใช้งาน Interrupt

1. IE : Interrupt Enable Register

EA	-	ET2	ES	ET1	EX1	ET0	EX0

2. IP : Interrupt Priority

X	X	PT2	PS	PT1	PX1	PT0	PX0

แต่ถ้าหากเราไม่ได้กำหนดความสำคัญของ Interrupt ในตัวของ MCS-51 จะเรียงลำดับความสำคัญตามค่าต่อไปนี้ จากความสำคัญสูงสุดไปยังความสำคัญน้อยที่สุด ดังนี้ IE0,TF0,IE1,TF1 และ RI+TI

H1.3: Short Example Interrupt Program

Example1:External Interrupt 0

จงเขียนโปรแกรมที่เมื่อเกิดการ Interrupt ที่ External Interrupt 0 และทำให้ LED ที่ต่ออยู่ที่ P1.0 เกิดการ Toggle

```
            ORG     0000H
            JMP     START

            ORG     0003H
            JMP     EX0_ROUTINE

            ORG     0100H
START:      SETB    IT0
            MOV     IE,#10000001B
            JMP     $

EX0_ROUTINE: CPL    P1.0
            RETI

            END
```

EA	-	ET2	ES	ET1	EX1	ET0	EX0

Example2: Timer Interrupt 0

เขียนโปรแกรมเพื่อสร้างสัญญาณ Clock โดยใช้ Timer Interrupt 0 เป็นฐานเวลา

```
        ORG     0000H          ; Code Start at Address 0000H
        JMP     START          ; Go to Start Label

        ORG      000BH
        JMP     Timer0_Routine

        ORG     0100H          ; Code Start at Address 0000H
START:  MOV     TMOD,#01H      ; Timer0 Mode 1 :16Bit Timer
        MOV     TH0,#0DCH      ; Initial for Timer0 High Byte
        MOV     TL0,#00H       ; Initial for Timer0 Low Byte
        MOV     IE,#10000010B  ; Timer0 Interrupt Enable
        SETB    TR0            ; Start Timer0 Run
LOOP:   JMP     $

Timer0_Routine:
        MOV     TH0,#0DCH      ; Reloaded TH0
        MOV     TL0,#00H       ; Reloaded TL0
        CPL     P1.7           ; Complement P1.7
        RETI

        END
```

EA	-	ET2	ES	ET1	EX1	ET0	EX0

Example3: Serial Interrupt

เขียนโปรแกรมเพื่อรับรหัส ASCII จากพอร์ตอนุกรมถ้าเป็น ASCII “C” ให้ LED ที่ต่ออยู่ที่ P1.0 ที่สภาวะปกติมีการกระพริบให้หยุดกระพริบ

```
        ORG     0000H          ; Code Start at Address 0000H
        JMP     START          ; Go to Start Label

        ORG      0023H
        JMP     Serial_Intr

        ORG     0100H          ; Code Start at Address 0000H
START:  CALL    Initial_Serial
        MOV     IE,#10010000B  ; Serial Interrupt Enable

LOOP:   CPL     P1.7           ; Complement P1.7
        CALL    Delay_1Sec
        MOV     A,Rx_Data
        CJNE    A,#'C',LOOP    ; If ASCII C
        JMP     $              ; Not Blink
```

EA	-	ET2	ES	ET1	EX1	ET0	EX0

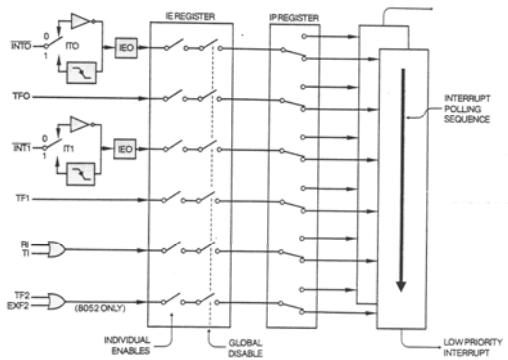
Example3: Serial Interrupt(ต่อ)

เขียนโปรแกรมเพื่อรับรหัส SACII จากพอร์ตอนุกรมถ้าเป็น ASCII “C” ให้ LED ที่ต่ออยู่ที่ P1.0 เกิดการ Toggle

```
Serial_Intr:  CLR     ES
              JB      TI,_X_Srl_Intr
              MOV     A,SBUF         ; Get Data
              MOV     SBUF,A         ; Echo Data
              MOV     Rx_Data,A
_X_Srl_Intr:  CLR     TI
              CLR     RI
              SETB    ES
              RETI

              END
```


H1.4: สรุป

1. การเปิดรับสัญญาณ Interrupt ต้องกำหนดโดย Register IE
2. เมื่อเกิด Interrupt โปรแกรมจะกระโดดไปทำงานที่ Interrupt Vectors ของแต่ละตัว และกลับมายังโปรแกรมหลักด้วย RETI



Interrupt Vector	Source
0003H	External INT0
000BH	Timer/Counter 0
0013H	External INT1
001BH	Timer/Counter 1
0023H	Serial Port
002BH	Timer/Counter 2*

EA	-	ET2	ES	ET1	EX1	ET0	EX0

H2: Timer Interrupts

TF0 Timer 0 Interrupt Vector

1 → jumps to 000BH

TF1 Timer 1 Interrupt Vector

1 → jumps to 001BH

TFx Interrupt

Interrupt Vector	Source
0003H	External INT0
000BH	Timer/Counter 0
0013H	External INT1
001BH	Timer/Counter 1
0023H	Serial Port
002BH	Timer/Counter 2*

Roll-over timer flag and interrupt

Polling: JNB TF, target

Example 11-2 **Example11.2**

Write a program that continuously gets 8-bit data from P0 and sends it to P1 while simultaneously creating a square wave of 200 μs period on pin P2.1. Use timer 0 to create the square wave. Assume that XTAL = 11.0592 MHz.

Solution:

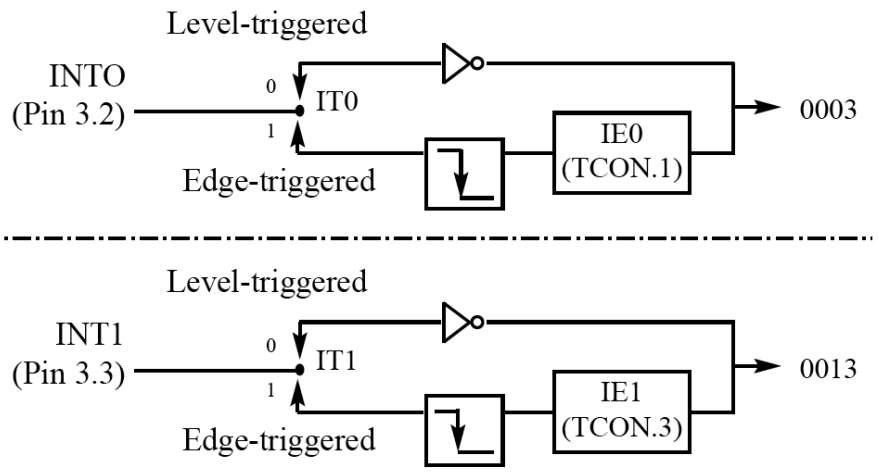
We will use timer 0 in mode 2 (auto reload). TH0 = 100/1.085 μs = 92.

```
;--Upon wake-up go to main, avoid using memory space
;allocated to Interrupt Vector Table
          ORG  0000H
          LJMP MAIN      ;by-pass interrupt vector table
;
;--ISR for Timer 0 to generate square wave
          ORG  000BH     ;Timer 0 interrupt vector table
          CPL  P2.1      ;toggle P2.1 pin
          RETI           ;return from ISR
;
;--The main program for initialization
          ORG  0030H     ;after vector table space
MAIN:     MOV  TMOD,#02H ;Timer 0,mode 2(auto reload)
          MOV  P0,#0FFH  ;make P0 an input port
          MOV  TH0,#-92  ;TH0=A4H for -92
          MOV  IE,#82H  ;IE=10000010 (bin) enable Timer 0
          SETB TR0       ;Start Timer 0
BACK:     MOV  A,P0      ;get data from P0
          MOV  P1,A      ;issue it to P1
          SJMP BACK      ;keep doing it
                         ;loop unless interrupted by TF0
          END
```


Lab83Tmr.Asm

```
Data            EQU             40H

                ORG             8000H
                JMP             Start

                ORG             9F42H
                JMP             Timer0_Intr

                ORG             8100H
Start:          MOV             TMOD,#00010001B     ; Timer0,1 Mode1 16Bit
                SETB            EA
                SETB            ET0
                SETB            TR0

Main_Loop:      MOV             A,Data
                MOV             P1,A                ; Out P1
                MOV             DPTR,#0E000H        ; Out Sound
                MOVX            @DPTR,A
                JMP             Main_Loop

Timer0_Intr:    CLR             TR0                 ; 1MC
                PUSH            DPH                 ; 2MC
                PUSH            DPL                 ; 2MC
                MOV             DPTR,#-374+21       ; 2MC
                MOV             TH0,DPH             ; 2MC
                MOV             TL0,DPL             ; 2MC
                SETB            TR0                 ; 1MC
                MOV             A,Data              ; 1MC
                CPL             A                   ; 1MC
                MOV             Data,A              ; 1MC
                POP             DPL                 ; 2MC
                POP             DPH                 ; 2MC
                RETI                                ; 2MC

                END
```

H3:External Interrupts



สัญญาณอินเตอร์รัปต์ที่เกิดจากภายนอกสามารถกำหนดให้มีการตรวจสอบได้ 2 แบบ คือ การตรวจสอบระดับสัญญาณ และจากการเปลี่ยนระดับสัญญาณโดยกำหนดได้จากการเซตหรือเคลียร์บิต IT1,IT0 (Interrupt Type Control Bit) ในรีจิสเตอร์ใช้งานเฉพาะ TCON ดังนี้

-บิต ITX (IT0 หรือ IT1) มีค่าเป็น 0 :สัญญาณอินเตอร์รัปต์ภายนอกจะถูกตรวจสอบจากระดับสัญญาณ โดยหากสถานะที่ขา INTX (INT0 หรือ INT1) มีค่าเป็น 0 จะทำให้เกิดสัญญาณอินเตอร์รัปต์และ บิต IEX (IE0 หรือ IE1) ถูกเซต

-บิต ITX มีค่าเป็น 1 : สัญญาณอินเตอร์รัปต์ภายนอกจะถูกตรวจสอบจากการเปลี่ยนระดับของสัญญาณ โดยหากตรวจสอบพบว่าที่ขา INTX เป็น 1 ในแมชชีนไซเคิลใด ๆ และเป็น 0 ในแมชชีนไซเคิลถัดไป บิต IEX ใน รีจิสเตอร์ TCON จะถูกเซตบิต IEX นี้จะทำให้เกิดการอินเตอร์รัปต์ซีพียูต่อไป

เนื่องจากสัญญาณอินเตอร์รัปต์จากภายนอกถูกตรวจสอบ 1 ครั้งในแต่ละแมชชีนไซเคิล ดังนั้นสัญญาณที่มีสถานะเป็น 1 หรือ 0 ควรจะคงค่าไว้ให้ได้อย่างน้อย 1 แมชชีนไซเคิล หรือ 12 คาบสัญญาณออสซิลเลเตอร์ ถ้าสัญญาณอินเตอร์รัปต์ภายนอกถูกตรวจสอบจากการเปลี่ยนระดับของสัญญาณ วงจรสร้างสัญญาณอินเตอร์รัปต์ ภายนอกจะคงค่าสถานะ 1 ไว้อย่างน้อย 1 ไซเคิล และคงสถานะ 0 ไว้อย่างน้อย 1 ไซเคิล ทั้งนี้เพื่อให้แน่ใจว่าสัญญาณอินเตอร์รัปต์ที่เกิดขึ้นจะตรวจสอบพบแน่นอนในกรณีนี้บิต IEX จะถูกเคลียร์โดยฮาร์ดแวร์เมื่อ MCS-51

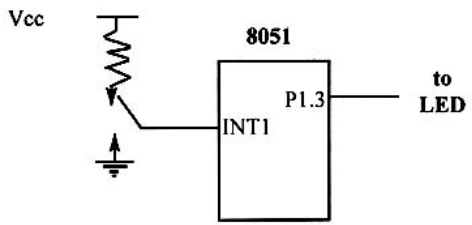
Example 11-5

Assume that the INT1 pin is connected to a switch that is normally high. Whenev goes low, it should turn on an LED. The LED is connected to P1.3 and is normally off. When it is turned on it should stay on for a fraction of a second. As long as the switch is pressed low, the LED should stay on.

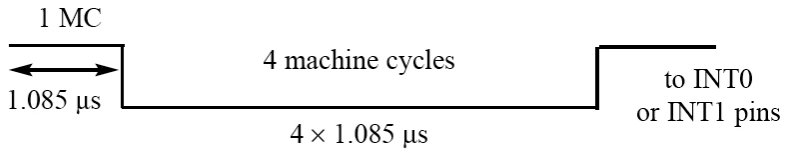
Example11-5

```
        ORG  0000H
        LJMP MAIN       ;by-pass interrupt vector table
;--ISR for hardware interrupt INT1 to turn on the LED
        ORG  0013H      ;INT1 ISR
        SETB P1.3       ;turn on LED
        MOV  R3,#255
BACK:   DJNZ R3,BACK    ;keep LED on for a while
        CLR  P1.3       ;turn off the LED
        RETI            ;return from ISR
;— MAIN program for initialization
        ORG  30H
MAIN:   MOV  IE,#10000100B  ;enable external INT 1
HERE:   SJMP HERE       ;stay here until get interrupted
        END
```


Pressing the switch will cause the LED to be turned on. If it is kept activated, the LED stays on.



Sampling the low level-triggered interrupt



Note: On RESET, IT0 (TCON.0) and IT1 (TCON.2) are both low, making external interrupts level-triggered.

Figure 11–5 Minimum Duration of the Low Level-Triggered Interrupt (XTAL = 11.0592 MHz)

Edge-triggered interrupts

D7							D0
TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

TF1	TCON.7	Timer 1 overflow flag. Set by hardware when timer/counter 1 overflows. Cleared by hardware as the processor vectors to the interrupt service routine.
TR1	TCON.6	Timer 1 run control bit. Set/cleared by software to turn timer/counter 1 on/off.
TF0	TCON.5	Timer 0 overflow flag. Set by hardware when timer/counter 0 overflows. Cleared by hardware as the processor vectors to the service routine.
TR0	TCON.4	Timer 0 run control bit. Set/cleared by software to turn timer/counter 0 on/off.
IE1	TCON.3	External interrupt 1 edge flag. Set by CPU when the external interrupt edge (H-to-L transition) is detected. Cleared by CPU when the interrupt is processed. *Note:* This flag does not latch low-level triggered interrupts.
IT1	TCON.2	Interrupt 1 type control bit. Set/cleared by software to specify falling edge/low-level triggered external interrupt.
IE0	TCON.1	External interrupt 0 edge flag. Set by CPU when external interrupt (H-to-L transition) edge is detected. Cleared by CPU when interrupt is processed. *Note:* This flag does not latch low-level triggered interrupts.
IT0	TCON.0	Interrupt 0 type control bit. Set/cleared by software to specify falling edge/low-level triggered external interrupt.

Lab81Ex.Asm

```
Data_Count:     EQU     40H
                ORG     8000H
                JMP     START
                ORG     9F45H
                JMP     EX1_ROUTINE
                ORG     8100H
START:          SETB    IT1             ; Adge Trigger
                MOV     IE,#10000100B
                MOV     Data_Count,#0
Main_Loop:      MOV     A,Data_Count
                CALL    Show_7Segment
                JMP     Main_Loop
;==========================================
EX1_ROUTINE:    PUSH    B
                PUSH    Acc
                INC     Data_Count
                CALL    Debounce
                CALL    Debounce
                CLR     IE1             ; Clear Interrupt External_0
                POP     Acc
                POP     B
                RETI                    ; Return from Interrupt
;==========================================
Debounce:       CLR     A
                MOV     B,A
_Dly00:         DJNZ    Acc,_Dly00
                DJNZ    B,_Dly00
                RET
;==========================================
Show_7Segment:  ANL     A,#0FH          ; Only 0-F
                MOV     DPTR,#CODE_TABLE
                MOVC    A,@A+DPTR
                MOV     DPTR,#0E060H
                MOVX    @DPTR,A
                RET
CODE_TABLE:     DB      3FH,06H,5BH,4FH,66H,6DH,7DH,07H
                DB      7FH,6FH,77H,7CH,79H,5EH,79H,71H
                END
```


Lab82Wrk.Asm

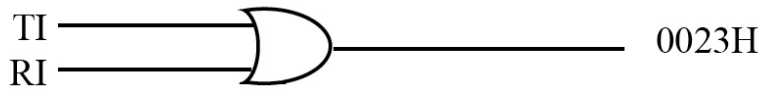
```
Data            EQU             20H

                ORG             8000H
                JMP             Start

                ORG             9F45H
                JMP             Ext_Int1

                ORG             8100H
Start:          MOV             IE,#10000100B       ; Enable All and EX1
                SETB            IT1                 ; 1=Adge Triger
                MOV             Data,#0             ; Init Value
Main_Loop:      MOV             DPTR,#0E060H
                MOV             A,Data
                JNB             Acc.0,JOB_1
                JB              Acc.0,JOB_2
                JMP             Main_Loop
;-------------------------------------------
Job_1:          MOV             A,#01111111B
                MOVX            @DPTR,A
                JMP             Main_Loop
;-------------------------------------------
Job_2:          MOV             A,#01111111B
                MOVX            @DPTR,A
                CALL            Delay
                MOV             A,#00000000B
                MOVX            @DPTR,A
                CALL            Delay
                JMP             Main_Loop
```

Lab82Wrk.Asm(2)

```
;==========================================
Delay:          PUSH        B
                PUSH        Acc
                MOV         B,#00H
                MOV         A,#00H
_Delay00:       DJNZ        Acc,_Delay00
                DJNZ        B,_Delay00
                POP         Acc
                POP         B
                RET
;==========================================
Ext_Int1:       PUSH        Acc
                MOV         A,Data
                CPL         A
                MOV         Data,A
                CALL        Delay
                CALL        Delay
                CLR         IE1             ; Clear Interrupt External_1
                POP         Acc
                RETI                        ; Return from Interrupt
                END
```


H3:Serial Interrupts

```
Serial = RI and TI Flag
    JNB RI,$ ; wait until rx complete
    JNB TI,$ ; wait until tx complete
```



Serial interrupt is invoked by TI or RI flags

Clearing RI and TI before the RETI instruction

CLR TI ; CLR RI

Example11-9

Example 11-9

Write a program in which the 8051 gets data from P1 and sends it to P2 continuously while incoming data from the serial port is sent to P0. Assume that XTAL = 11.0592. Set the baud rate at 9600.

Solution:


```
        ORG 0
        LJMP MAIN
        ORG 23H
        LJMP SERIAL     ;jump to serial ISR
        ORG 30H
MAIN:   MOV P1,#0FFH    ;make P1 an input port
        MOV TMOD,#20H   ;timer 1, mode 2 (auto reload)
        MOV TH1,#0FDH   ;9600 baud rate
        MOV SCON,#50H   ;8-bit,1 stop, REN enabled
        MOV IE,#10010000B   ;enable serial interrupt
        SETB TR1            ;start timer 1
BACK:   MOV  A,P1           ;read data from port 1
        MOV  P2,A           ;send it to P2
        SJMP BACK           ;stay in loop indefinitely
;-----------------SERIAL PORT ISR
        ORG 100H
SERIAL: JB  TI,TRANS        ;jump if TI is high
        MOV A,SBUF          ;otherwise due to receive
        MOV P0,A            ;send incoming data to P0
        CLR RI              ;clear RI since CPU doesn't
        RETI                ;return from ISR
TRANS:  CLR TI              ;clear TI since CPU doesn't
        RETI                ;return from ISR
        END
```

Table 11–2 :Interrupt Flag Bits for the 8051/52

Interrupt	Flag	SFR Register Bit
External 0	IE0	TCON.1
External 1	IE1	TCON.3
Timer 0	TF0	TCON.5
Timer 1	TF1	TCON.7
Serial port	T1	SCON.1
Timer 2	TF2	T2CON.7 (AT89C52)
Timer 2	EXF2	T2CON.6 (AT89C52)

Lab84Srl.Asm

```
Rx_Data         EQU             40H

                ORG             8000H
                JMP             8100H

                ORG             9F4BH
                JMP             Serial_Intr

                ORG             8100H
                CALL            Delay
                MOV             SCON,#50H           ; Mode1 Serial Port
                MOV             RCAP2H,#0FFH        ; Reload values for desired
                MOV             RCAP2L,#0DCH        ; Baud rate 9600@11.0592
                MOV             T2CON,#34H          ; Turn on Timer 2
                MOV             IE,#10010000B       ; Enable All and Serial
                MOV             A,#0CH              ; Clear Screen
                MOV             SBUF,A              ; Send ClearScreen
                JNB             TI,$
                MOV             Rx_Data,#'A'        ; Init Value

Main_Loop:      MOV             A,Rx_Data
                CALL            SegShow_2HEX
                JMP             Main_Loop

;==================================================
Delay:          CLR             A
                MOV             B,A
_Dly00:         DJNZ            Acc,_Dly00
                DJNZ            B,_Dly00
                RET
```


Lab84Srl.Asm(2)

```
;=== Show 2HEX @Acc to 7_Segment Display ===
SegShow_2HEX:   PUSH            Acc
                SWAP            A
                MOV             B,#2            ; Digit 2
                CALL            SegShow_1HEX
                POP             Acc
                MOV             B,#3            ; Digit 3
SegShow_1HEX:   XCH             A,B
                MOV             DPTR,#0E002H    ; Digit Show
                MOVX            @DPTR,A
                XCH             A,B
                ANL             A,#0FH
                MOV             DPTR,#Code_Segment
                MOVC            A,@A+DPTR
                MOV             DPTR,#0E001H    ; Position Show
                MOVX            @DPTR,A
                CLR             A
                DJNZ            Acc,$
                RET
Code_Segment:   DB              3FH,06H,5BH,4FH,66H,6DH,7DH,07H
                DB              7FH,6FH,77H,7CH,79H,5EH,79H,71H

;=================================================================
;  Serial Interrupt Service Program
;=================================================================
Serial_Intr:    CLR             ES
                JB              TI,_X_Srl_Intr
                MOV             A,SBUF          ; Get Data
                MOV             SBUF,A          ; Echo Data
                MOV             Rx_Data,A
_X_Srl_Intr:    CLR             TI
                CLR             RI
                SETB            ES
                RETI

                END
```


H4:Interrupt priority

Highest to Lowest Priority

External Interrupt 0	(INT0)
Timer Interrupt 0	(TF0)
External Interrupt 1	(INT1)
Timer Interrupt 1	(TF1)
Serial Communication	(RI + TI)
Timer 2 (8052 only)	**TF2**

Interrupt priority upon reset

Figure 11–8: Interrupt Priority Register (Bit-addressable)

D7							D0
--	--	PT2	PS	PT1	PX1	PT0	PX0

Priority bit = 1 assigns high priority. Priority bit = 0 assigns low priority.

--	IP.7	Reserved
--	IP.6	Reserved
PT2	IP.5	Timer 2 interrupt priority bit (8052 only)
PS	IP.4	Serial port interrupt priority bit
PT1	IP.3	Timer 1 interrupt priority bit
PX1	IP.2	External interrupt 1 priority bit
PT0	IP.1	Timer 0 interrupt priority bit
PX0	IP.0	External interrupt 0 priority bit

User software should never write 1s to unimplemented bits, since they may be used in future products.

Setting interrupt priority with the IP register

Example11-13

Example 11-13

Assume that after reset, the interrupt priority is set by the instruction "`MOV IP,#00001100B`". Discuss the sequence in which the interrupts are serviced.

Solution:

The instruction "`MOV IP,#00001100B`" (B is for binary) sets the external interrupt 1 (INT1) and timer 1 (TF1) to a higher priority level compared with the rest of the interrupts. However, since they are polled according to Table 11-3, they will have the following priority.

Highest Priority	External Interrupt 1	(INT1)
	Timer Interrupt 1	(TF1)
	External Interrupt 0	(INT0)
	Timer Interrupt 0	(TF0)
Lowest Priority	Serial Communication	(RI + TI)